PROJECT-BASED LEARNING

PHASE-II EVALUATION

RESQNET: Smart Urban Emergency & Transportation Network Simulator

Team ID: DSCPP-III-2026-T070

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID:	DSCPP-III-2026-T070
Team Name:	RESQNET
Semester:	3rd
Section:	BTECH
Domain:	Graph Algorithms & DSA

Team Members

1. Chhavi – 2510012506
2. Tapashya – 2510011649
3. Arpit Chahal – 2510012000

Mentor

Mr. Darshan Singh

Interactions completed: 6

Phase-I Recap & Feedback Incorporated

Phase-I Status

Finalized the problem statement, identified disaster and vehicle types, modeled the city network as a weighted graph, and planned the priority queue and Dijkstra routing architecture.

Feedback Received

- Build a role-based login and web interface
- Expand the fleet to support all specialized rescue vehicles
- Implement dynamic road closures and shortest-path dispatch

Changes Incorporated

- Built user and dispatcher portals with interactive 10-junction map
- Configured 80 response vehicles across 25 emergency categories
- Implemented Dijkstra routing, road-closure toggles, and ETA engine

Problem Statement & Finalized Objectives

Problem Statement

Design a multi-disaster city simulation that prioritizes concurrent emergencies, dispatches optimal response vehicles, and dynamically reroutes around road closures using Data Structures.

Finalized Objectives

1. **Incident Triage:** Support 25 emergency types and rank pending reports by priority score ($10 \times \text{severity} + 0.5 \times \text{people}$).
2. **Weighted City Graph:** Model 10 junctions and 14 roads with Dijkstra shortest-path routing and road-closure bypass.
3. **Multi-Vehicle Allocation:** Match required unit types from an 80-vehicle fleet based on fastest travel time (ETA).
4. **Interactive Portal:** Provide role-based Citizen Reporting and Dispatcher Control views synced with the core engine.

Updated System Architecture / Workflow



Major Components

- **Priority & Store:** Incident queue, severity scoring, user auth
- **Graph & Fleet:** 10-junction city graph, 80-vehicle fleet pool
- **Web Portal:** Citizen reporting & Dispatcher control room

Changes Since Phase-I

- Expanded from 7 planned disasters to 25 emergency categories
- Added direct air-routing (16.7 m/s) for Helicopter units
- Added interactive map with live status & route confirmation

Module Design: Phase 1 – Incident & Resource Module

Phase 1 Modules

1. `common.h`: Shared constants, 25 incident & 8 vehicle enums
2. `incident.h/c`: Incident store, priority score, pending lookup
3. `types.h/c`: Vehicle requirements & safety precautions
4. `fleet.h/c`: 80-unit fleet loader from `data/vehicles.txt`
5. `auth.h/c` **&** `io.h/c`: Role authentication & input validation

Functional Requirements

- Record emergency type, junction (1–10), severity, and people count
- Calculate priority score
  $\text{Score} = 10.0 \times \text{severity} + 0.5 \times \text{people}$
- Provide instant incident-specific safety precautions to reporters
- Track incident lifecycle (`REPORTED`, `DISPATCHED`, `RESOLVED`)

Module Design: Phase 2 – Graph Routing & Dispatch

Phase 2 Modules

1. `city.h/c`: Weighted graph, Dijkstra algorithm, road closures
2. `dispatch.h/c`: `vehicle_eta()`, `find_best()`, `dispatch_incident()`
3. `main.c`: User & Dispatcher workflows and engine integration
4. **Web Portal:** React 19 Citizen & Dispatcher control interfaces

Functional Requirements

- Compute shortest travel time (seconds) across 14 bidirectional roads
- Bypass `BLOCKED` roads dynamically and recompute alternate paths
- Route Helicopters via straight-line distance ignoring road blocks
- Lock assigned vehicles until the dispatcher resolves the incident

Code Pipeline – Phase 1 & Phase 2

Phase 1 – Incident Reporting & Priority Pipeline

`user_report()` ⟶ `store_add()` ⟶ `priority_score()` ⟶ `precaution_for()` ⟶ `store_save()` ⟶ `data/incidents.txt` (`REPORTED`)

Phase 2 – Dijkstra Routing & Fleet Dispatch Pipeline

`store_best_pending()` ⟶ `needs_for()` ⟶ `find_best()` ⟶ `city_shortest_path()` / `heli_seconds()` ⟶ `dispatch_incident()` ⟶ `DISPATCHED`

Each incident is assigned up to 4 nearest available units; marking an incident `RESOLVED` releases its vehicles back to the available pool.

Technology Stack & Development Methodology

Technology Stack

- **Core Engine:** C / C++ (`gcc -O2 -Wall -Wextra`)
- **Data Structures:** Weighted Graph, Priority Queue, Structs
- **Algorithms:** Dijkstra Shortest Path, Euclidean Air-Routing
- **Web Portal:** React 19, TypeScript, Vite, Tailwind CSS, Supabase
- **Version Control:** Git / GitHub

Development Methodology

1. Requirement Analysis & Graph Modeling
2. Modular Header & Struct Design
3. Priority Queue & Fleet Loader Implementation
4. Dijkstra Routing & Dispatcher Integration
5. Scenario Testing & Web Portal Verification

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures	Weighted Graph, Priority Selection, Dynamic Arrays, Lookup Tables	`city.h/c`, `incident.h/c`, `fleet.h/c`, `types.h/c`
Modular Design	Structured ADTs, Encapsulation, Header Guards, Enum State Machines	`common.h`, `dispatch.h/c`, `auth.h/c`, `io.h/c`
Algorithms	Dijkstra Shortest Path, Greedy Unit Selection, Euclidean Air ETA	`city_shortest_path()`, `find_best()`, `vehicle_eta()`
File Handling	Persistent Incident Store, Fleet Config, City Graph & User Records	`city.txt`, `vehicles.txt`, `incidents.txt`, `users.txt`

Development & Implementation Progress

Module	Status	Work Completed
Phase 1 – City Graph & Fleet Pool	**Completed**	`city.h/c` (10 junctions, 14 roads), `fleet.h/c` (80 response units)
Phase 1 – Incident & Priority Engine	**Completed**	`incident.h/c`, `types.h/c` (25 emergency types & precautions)
Phase 2 – Dijkstra & Fleet Dispatch	**Completed**	`dispatch.h/c` (shortest-path ETA, air routing, road closures)
Phase 2 – Role-Based Web Portal	**Completed**	Citizen reporting & Dispatcher control views with live status sync
Phase 3 – Resilience & Multi-City	**Pending**	Articulation-point resilience analysis and hospital bed capacity

Working Prototype: Phase 1 – User Reporting

Implemented Features

- **Live Web Portal:** `spline-stall-88999147.figma.site`
- **Role Authentication:** Separate Citizen (`U`) and Dispatcher (`D`) login
- **25 Emergency Types:** Quick-pick situations & junction selector
- **Live Precautions:** Shows assigned units, routes, ETAs & safety steps

Phase 1 CLI Output (`./resqnet`)

```
Welcome taps (User)
--- Your current reports ---
------------------------------------------------------------
 #1: Fire Emergency at Main Road
 Status: DISPATCHED
 Your note: Vehicle on fire or smoke
  [GO] FireTruck-F2 (Fire Truck) from Base Station
       Route   : Base Station -> Main Road
       ETA     : 60 s (about 1 min)
  [GO] Ambulance-A3 (Ambulance) from City Hospital
       Route   : City Hospital -> Station Chowk -> Main Road
       ETA     : 90 s (about 2 min)
 Precautions until the team arrives:
   Leave the area and keep others away. Do not use lifts.
```

Working Prototype: Phase 2 – Dispatcher Engine

Implemented Features

- **Priority Ordering:** Scores incidents (92.0 vs 81.5) for next dispatch
- **Dijkstra Shortest Path:** Computes junction-by-junction route & ETA
- **Multi-Unit Match:** Dispatches Fire Truck, Ambulance, and Police
- **Road Status Control:** Toggles any of the 14 city roads open/`BLOCKED`

Phase 2 CLI Output (`./resqnet`)

```
Welcome Chhavi (Dispatcher)
------------------------------------------------------------
 #2: Gas Explosion at Park Lane  [DISPATCHED]
 Severity 9/10 | People 4 | Priority 92.0 | Reported by taps
  [GO] FireTruck-F1 (Fire Truck) from Base Station
       Route   : Base Station -> Park Lane
       ETA     : 50 s (about 1 min)
  [GO] Ambulance-A1 (Ambulance) from City Hospital
       Route   : City Hospital -> Civil Lines -> Bus Stand
                 -> Market Square -> Park Lane
       ETA     : 180 s (about 3 min)
  [GO] Police-P5 (Police) from Park Lane | ETA: 0 s
------------------------------------------------------------
```

Testing, Results & Validation

Module	Expected Result	Observed Result	Status
`city.c` (Dijkstra)	Compute fastest path & bypass closed roads	Optimal path & alternate reroute verified	**Pass**
`incident.c` (Priority)	Select highest score ($10S + 0.5P$) first	Priority 92.0 selected before 81.5	**Pass**
`fleet.c` & `types.c`	Load fleet & map required vehicle types	80 vehicles loaded; exact needs matched	**Pass**
`dispatch.c`	Assign nearest free units & calculate ETA	Closest available units assigned with ETA	**Pass**
`auth.c` & Web UI	Enforce role access & sync dispatch status	User & Dispatcher views synced accurately	**Pass**

GitHub Repository & Development Activity

Repository Information

- **GitHub:** `mchhavi929-ship-it/Resqnet`
- **Live Demo:** `spline-stall-88999147.figma.site`
- **README.md:** Updated with architecture & setup guide
- **Contributors:** Chhavi, Tapashya, Arpit Chahal
- **Branch:** `main`

Development Activity

- Structured into `include/`, `src/`, `data/`, and `frontend/`
- Modular commits across graph, fleet, priority, and UI
- Verified end-to-end reporting and dispatch workflows
- Individual module ownership tracked in repository
- Complete documentation and datasets included

Individual Contribution

Member	Role	Technical Contribution
Chhavi – 2510012506	Team Lead	Built `city.h/c` (graph, Dijkstra, road closures), `dispatch.h/c` (ETA & route display), `common.h`, and integrated `main.c` & Web UI.
Tapashya – 2510011649	Developer	Built `fleet.h/c` (vehicle structs & fleet loader), `types.h/c` (incident vehicle needs), and coordinated `find_best()` & `dispatch_incident()`.
Arpit Chahal – 2510012000	Developer	Built `incident.h/c` (incident store, priority score, pending selection), `io.h/c` (input helpers), and `auth.h/c` with `data/*.txt`.

Challenges, Phase-III Roadmap & References

Challenges & Solutions

- **Blocked Road Rerouting:** Handled unreachable paths cleanly and switched to alternate Dijkstra routes
- **Busy Fleet Tracking:** Prevented double-booking by scanning active `DISPATCHED` incidents before allocation

Phase-III Roadmap

1. Articulation-point detection for critical city junctions
2. Hospital capability and bed-capacity matching
3. Multi-scenario stress testing and final evaluation demo

References

1. Cormen et al., *Introduction to Algorithms*
2. E. Balagurusamy, *Object Oriented Programming with C++*, 8th Ed.
3. Herbert Schildt, *The Complete Reference C++*, 4th Ed.
4. `cppreference.com` & React Documentation

Thank You
Questions & Suggestions